

Today 8:51 AM



Unified Engagement...
File



quote-followup-ass...
File



Quote Followup Bra...
File



Measurement & Ana...
File



LinkedIn Execution ...
File



growth-experiment-...
File



Content Calendar & ...
File



Compliance Review ...
File



7-Day Lead Sprint O...
File

Target List Template Quote Followup

Unable to display
visualization

these were delivered by marketing agent as assets and full strategy plan for a campaign, how can i use it to streamline and automate the process.. give me ideas

The top repositories to use depend on your stack and goals:

- [fire Enrich](#): An AI-powered data enrichment tool that transforms raw data (like emails or domains) into rich datasets (funding, tech stacks, company profiles) using multi-agent LLMs and Firecrawl. [GitHub](#)
- **Crawl4AI**: Built entirely for AI workflows. It crawls, extracts JavaScript-heavy websites, cleans content into LLM-ready Markdown, and processes structured data using custom prompts. [LinkedIn · Mayank A. · 1](#)
- **Firecrawl**: An open-source web data API that converts sites into clean Markdown or structured JSON. It features LLM-based extraction without needing to maintain complex CSS selectors. [LinkedIn · Mayank A. · 2](#)
- [Crawlee](#): The premier Node.js/TypeScript web scraping and browser automation library. It handles proxy rotation, handles both HTTP and headless browsers, and is optimized to build reliable crawlers for RAG pipelines. [GitHub](#)

check some of these and how do they fit



so what is missing? what are my next steps currently what is build so far is the strategist_bundle.md, and i already have the nexus sprint running with the following roster (attahced)

Based on the recommended blueprint, the next deliverable should be a **verified end-to-end integration plan**, not another conceptual architecture.

To build it accurately, I need the current state of both systems.

Please send me:

1. Governance Kernel repository

- GitHub URL, or
- repository tree plus the key files, especially:
 - STRATEGIST_BUNDLE.md
 - ENGINE_STATE.md
 - database migrations
 - package/module structure
 - n8n workflows
 - integration clients
 - Docker configuration
 - tests

2. Agency Agents / Nexus setup

- Whether you use the upstream msitarzewski/agency-agents repository, a fork, or copied agents
- The exact Nexus activation prompt
- Current output location and file format
- One completed sprint output, if available

3. Current operating systems

Confirm what you actually intend to use for:

- CRM: Pardot/Salesforce, HubSpot, Dynamics, or another system
- Email outreach
- Nurture automation
- Booking: Outlook Bookings or another tool

- Database
- Orchestration
- Hosting

4. Initial deployment objective

- Internal campaign execution tool
- Customer-facing SaaS
- Internal first, SaaS later

Once I have that, I will structure the implementation into these independent streams:

Stream	Responsibility
A. Strategy Production	Nexus sprint, agent outputs, synthesis
B. Bundle Contract	Strategy output schema, provenance, versioning
C. Governance Kernel	State, approvals, compliance, audit
D. Campaign Ingestion	Convert approved bundle into executable campaign objects
E. Lead Operations	Import, deduplication, scoring, enrichment
F. Channel Execution	Email, nurture, LinkedIn-assisted actions
G. Engagement Operations	Replies, routing, SLA, booking
H. Analytics and Experiments	Events, attribution, variants, recommendations
I. Operator Interface	Review, approvals, monitoring
J. Platform Operations	Deployment, security, observability, backups

For every stream, the blueprint will identify:

Existing implementation

- Missing functionality
- Reusable open-source repository/tool
- Required custom code
- Integration interface
- Dependencies
- Acceptance tests
- MVP / later phase



The immediate technical target will be:

Nexus sprint



- versioned strategist output
- validated handoff contract
- Governance Kernel ingestion
- campaign in review state
- human approval
- lead import and scoring
- governed channel execution
- reply and SLA processing
- analytics feedback

Send the Governance Kernel repository first. That will let me assess what is genuinely built, what is only scaffolded, and where proven repositories should be integrated rather than duplicated.

i will use you in opencode, in my copy and rebranded project folder where i use these agents also i will put in the same folder a pdf of this chat history, i just need you to help me with any instructions or files that you recommend i add so you can straight away understand and continue rather than analyse and interpret. give me also recommendations where i should put the kernel files backend/

Use a **single repository**, but keep the strategy layer, governance kernel, integrations, workflows, and UI as clearly separated streams.

I prepared a starter structure with the context files and instructions needed for OpenCode to understand the project immediately:

Download the CampaignOps project bootstrap

Recommended root structure

```
your-project/  
├── AGENTS.md  
├── README.md  
├──  
├── .opencode/  
│   └── agents/  
│       └── your rebranded Agency Agents  
├──  
└──
```



```

├─ strategy/
│   ├── briefs/
│   ├── bundles/
│   ├── schemas/
│   ├── templates/
│   └─ raw_outputs/
│
├─ backend/
│   ├── pyproject.toml
│   ├── src/
│   │   └─ campaignops/
│   └─ tests/
│
├─ workflows/
│   └─ n8n/
│
├─ frontend/
├─ infra/
│
└─ docs/
    ├── PROJECT_CONTEXT.md
    ├── ARCHITECTURE.md
    ├── CURRENT_STATE.md
    ├── INTEGRATION_MAP.md
    ├── DECISIONS.md
    ├── WORKING_WITH_OPENCODE.md
    └─ reference/
        └─ chat-history.pdf

```

Where to put the kernel

Place the kernel under:

backend/src/campaignops/



Do not place all files directly inside `backend/`.

Recommended structure:

```

backend/
├─ pyproject.toml
├─ src/
│   └─ campaignops/
│       ├── api/
│       │   |
│       │   └─ kernel/
│       │       ├── state_machine.py
│       │       ├── policies.py
│       │       ├── approvals.py
│       │       └─ audit.py

```



```

|   |   | ingestion/
|   |   | | bundle_loader.py
|   |   | | validators.py
|   |   | | importer.py
|   |   |
|   |   | campaigns/
|   |   | leads/
|   |   | scoring/
|   |   | enrichment/
|   |   | compliance/
|   |   |
|   |   | channels/
|   |   | | email/
|   |   | | nurture/
|   |   | | linkedin_tasks/
|   |   |
|   |   | engagement/
|   |   | | replies/
|   |   | | sla/
|   |   | | booking/
|   |   |
|   |   | analytics/
|   |   | experiments/
|   |   |
|   |   | integrations/
|   |   | | crm/
|   |   | | booking/
|   |   | | research/
|   |   | | enrichment/
|   |   | | email/
|   |   | | llm/
|   |   |
|   |   | shared/
|   |   | | models.py
|   |   | | errors.py
|   |   | | config.py
|   |
|   | tests/
|   | | unit/
|   | | integration/
|   | | e2e/

```

What belongs in `kernel/`

Only the central governance logic:

- valid state transitions;
- campaign and lead execution eligibility;
- policy decisions;
- approval requirements;
- audit-event creation;

- cross-domain governance.

Do not move every business module into `kernel/`.

For example:

```
Lead scoring      → scoring/
Company enrichment → enrichment/
Compliance rules  → compliance/
Reply classification → engagement/replies/
SLA calculation   → engagement/sla/
Experiment analysis → experiments/
```



The kernel coordinates these modules but does not absorb them.

How to separate the streams

Stream 1 — Strategy Studio

Location:

```
.opencode/agents/
strategy/
```



Contains:

- rebranded Agency Agents;
- Nexus Sprint activation;
- campaign briefs;
- specialist outputs;
- `STRATEGIST_BUNDLE.md` ;
- structured campaign artifacts;
- provenance and approvals.

Recommended campaign structure:

```
strategy/bundles/quote-followup-v1/
├─ STRATEGIST_BUNDLE.md
├─ campaign_spec.yaml
├─ icp_segments.yaml
├─ message_matrix.yaml
├─ measurement_plan.yaml
├─ compliance_review.yaml
├─ sprint_manifest.yaml
├─ approval_record.yaml
├─
├─ assets/
└─ raw_outputs/
```



The strategy stream produces campaign intent and assets. It never executes campaigns directly.

Stream 2 — Bundle Contract and Ingestion

Location:

```
strategy/schemas/  
backend/src/campaignops/ingestion/
```



Contains:

- YAML/JSON schemas;
- bundle loader;
- required-field validation;
- version compatibility;
- asset-reference validation;
- idempotent importer.

This is the bridge between the Nexus Sprint and the kernel.

```
Nexus outputs  
→ structured bundle  
→ validator  
→ importer  
→ campaign in ready_for_review
```



Stream 3 — Governance Kernel

Location:

```
backend/src/campaignops/kernel/  
backend/src/campaignops/compliance/
```



Contains:

- state machines;
- approval policies;
- compliance decisions;
- execution eligibility;
- audit;
- suppression enforcement.

This is your core proprietary layer.

Stream 4 — External integrations

Location:

backend/src/campaignops/integrations/



Use adapters rather than embedding provider-specific code throughout the kernel.

For example:

Python



Run

```
class EmailProvider:
    def create_sequence(...)
    def add_contact(...)
    def stop_contact(...)
    def get_events(...)
```

Implementations could be:

```
SmartleadEmailProvider
InstantlyEmailProvider
DensheesEmailProvider
OutlookEmailProvider
```



The same principle applies to:

- CRM;
- nurture;
- booking;
- company research;
- enrichment;
- LLM providers.

Stream 5 — n8n orchestration

Location:

workflows/n8n/



Keep n8n workflows outside the backend package.

Example:

```
01_bundle_ingestion.json
02_target_import.json
03_draft_generation.json
04_compliance_and_approval.json
05_email_execution.json
06_reply_processing.json
07_sla_monitor.json
08_daily_summary.json
```



n8n should:

- receive events;
- invoke backend APIs;
- call integrations;
- schedule workflows;
- handle notifications.

n8n should not own:

- campaign state;
- lead state;
- approval truth;
- compliance truth;
- business transition rules.

Those remain in Postgres and the backend.

Files OpenCode should always read first

The most important file is:

```
AGENTS.md
```



OpenCode agents should read it before making changes.

The included version tells the coding agent:

- what the two systems are;
- where each responsibility belongs;
- not to introduce agent-to-agent runtime messaging;
- not to redesign existing kernel functionality;
- to reuse proven integrations;
- to inspect current state before coding;
- not to claim completion without tests.

The supporting files are:

`docs/PROJECT_CONTEXT.md`

Explains the product, objectives, boundaries, and MVP philosophy.

`docs/ARCHITECTURE.md`

Defines all streams and repository ownership.

`docs/CURRENT_STATE.md`

This is especially important. It should always reflect what is genuinely:

- implemented;
- partially implemented;
- scaffolded;
- tested;
- missing.

Update this after every meaningful development session.

`docs/INTEGRATION_MAP.md`

Prevents rebuilding commodity capabilities.

It records:

```
Capability
→ build / integrate / reuse
→ selected tool
→ adapter interface
→ alternative provider
```



`docs/DECISIONS.md`

Records architectural decisions so future agents do not reopen settled questions.

`docs/WORKING_WITH_OPENCODE.md`

Contains the startup prompt to use in a new OpenCode session.

Where to put the PDF of this chat

Use:

`docs/reference/chat-history.pdf`



Do not make the PDF the primary source of project truth.

The PDF is too long and contains evolving discussions, corrections, and discarded options. A coding agent may interpret outdated parts as current decisions.

Use this priority order:

1. AGENTS.md
2. docs/CURRENT_STATE.md
3. docs/DECISIONS.md
4. docs/ARCHITECTURE.md
5. docs/INTEGRATION_MAP.md
6. chat-history.pdf as historical reference



Recommended OpenCode startup instruction

Use this at the beginning of a new session:

You are working inside the CampaignOps repository.



Before coding:

1. Read AGENTS.md.
2. Read docs/PROJECT_CONTEXT.md, docs/ARCHITECTURE.md, docs/CURRENT_STATE.md, docs/INTEGRATION_MAP.md, and docs/DECISIONS.md.
3. Inspect the relevant existing implementation and tests.
4. State what already exists, what is missing, and which architectural stream owns the requested work.
5. Reuse existing modules and proven integrations.
6. Do not create agent-to-agent runtime communication.
7. Keep Postgres as the source of truth.
8. Do not redesign the governance kernel without explicit approval.
9. Do not claim completion without tests or a verified execution path.
10. Update docs/CURRENT_STATE.md after completing work.

What you should do when copying the existing kernel

1. Copy the kernel into:

backend/src/campaignops/



2. Preserve its current internal modules initially. Do not reorganize everything immediately.
3. Fill in docs/CURRENT_STATE.md using actual repository evidence.
4. Map existing files to the target structure.
5. Refactor only where there is a concrete problem, such as:

- circular imports;
 - provider-specific logic inside domain modules;
 - missing package boundaries;
 - duplicate state ownership;
 - untestable n8n-only business logic.
6. Add integration adapters around existing functionality instead of rewriting it.
 7. Make the first end-to-end milestone:

Nexus Sprint

- approved strategist bundle
- validated ingestion
- campaign created
- target list imported
- scoring
- compliance
- approval
- one controlled email sequence
- reply
- SLA
- analytics reconciliation



That should be the first complete vertical slice before expanding autonomy or adding more channels. `</>`

give me the instructions in an .md file

I've prepared the content below as a Markdown document. Save it as:

docs/OPENCODE_BOOTSTRAP.md



or place it in the repository root as:

OPENCODE_BOOTSTRAP.md



`</>` Markdown



OpenCode Bootstrap Instructions

Purpose

This repository combines two independent systems that work together:

1. **Strategy Studio**
 - Rebranded Agency Agents
 - Nexus Sprint
 - Campaign planning
 - Multi-agent collaboration
 - Creative assets
2. **CampaignOps Kernel**
 - Deterministic backend
 - Governance
 - State machine
 - Compliance
 - Approval
 - Campaign execution
 - Analytics
 - Audit

The Strategy Studio creates campaigns.

The CampaignOps Kernel executes them safely.

These systems are intentionally separated.

Repository Layout

...

/

```
|— .opencode/
|   |— agents/
|
|— strategy/
|   |— briefs/
|   |— bundles/
|   |— raw_outputs/
|   |— templates/
|   |— schemas/
|
|— backend/
|   |— src/
|       |— campaignops/
|           |— kernel/
|           |— ingestion/
|           |— campaigns/
|           |— leads/
|           |— scoring/
|           |— enrichment/
|           |— compliance/
```

```
| | | | channels/
| | | | engagement/
| | | | analytics/
| | | | experiments/
| | | | integrations/
| | | | shared/
| | |
| | | | tests/
|
| | workflows/
| | | n8n/
|
| | frontend/
|
| | infra/
|
| | docs/
|
| | ...
```

Read Order

Every coding session should begin by reading:

1. AGENTS.md
2. docs/PROJECT_CONTEXT.md
3. docs/CURRENT_STATE.md
4. docs/ARCHITECTURE.md
5. docs/INTEGRATION_MAP.md
6. docs/DECISIONS.md

Only use the conversation history PDF as historical reference.

The Markdown documents are authoritative.

Core Architecture

...

Campaign Brief

↓

Nexus Sprint

↓

STRATEGIST_BUNDLE

↓

Bundle Validation

↓

CampaignOps Ingestion

↓

Campaign State

↓

Lead State

↓

Compliance

↓

Approval

↓

Execution

↓

Replies

↓

SLA

↓

Analytics

...

Architectural Principles

Strategy is NOT execution

The Nexus Sprint generates:

- campaign strategy
- messaging
- content
- assets

- experiments
- measurement plans

It never executes campaigns.

CampaignOps owns execution

CampaignOps owns:

- state
- approvals
- compliance
- execution
- audit
- analytics
- lifecycle

Source of Truth

Postgres is the source of truth.

n8n orchestrates workflows.

External systems never own campaign state.

Runtime Philosophy

Avoid runtime agent-to-agent communication.

Logical agents are implementation modules.

Prefer:

- deterministic Python
- explicit state machines
- policy engines
- adapters
- shared state

Use LLMs only where semantic reasoning is required.

Current LLM Responsibilities

LLMs should normally be limited to:

- CampaignSpec extraction
- Message drafting
- Reply classification
- Narrative summaries

Everything else should be deterministic whenever possible.

Development Philosophy

Before writing new code:

1. Search the repository.
2. Reuse existing modules.
3. Reuse existing integrations.
4. Avoid duplicate functionality.
5. Prefer adapters over rewrites.

Open Source Philosophy

Always prefer integrating mature software before building.

Build

- Governance Kernel
- State Machines
- Compliance Engine
- Approval Engine
- Audit
- Experiment Logic

These are proprietary.

Integrate

- CRM
- Email
- Marketing Automation
- Booking
- Dashboards
- Enrichment
- Company Research

Current Preferred Stack

```
| Capability | Preferred |
|-----|-----|
| Database | PostgreSQL |
| Orchestration | n8n |
| CRM | Adapter (Pardot / Salesforce / HubSpot / Dynamics) |
| Email | Smartlead / Instantly / Denshees |
| Booking | Outlook Bookings |
| Marketing Automation | Mautic |
| Company Research | Firecrawl |
| Enrichment | Fire Enrich pattern / Apollo / Clay |
| Analytics | Metabase |
| LLM | OpenAI / OpenRouter |
```

Repository Streams

Stream A

Strategy Studio

Location

strategy/

Produces

- campaign brief
- strategist bundle
- assets
- messaging
- measurement

Stream B

Bundle Contract

Responsible for

- schemas
- validation
- ingestion

Stream C

CampaignOps Kernel

Responsible for

- governance
- state
- approvals
- audit
- compliance

Stream D

Lead Operations

Responsible for

- imports
- deduplication
- scoring
- suppression

Stream E

Channel Execution

Responsible for

- email
- nurture
- LinkedIn task generation

Stream F

Engagement

Responsible for

- replies
- SLA
- booking

Stream G

Analytics

Responsible for

- metrics
- experiments
- recommendations

Stream H

Infrastructure

Responsible for

- Docker
- n8n
- Postgres
- deployments
- backups

Bundle Contract

The Strategy Studio should eventually produce:

```

```
campaign_spec.yaml
icp_segments.yaml
message_matrix.yaml
measurement_plan.yaml
compliance_review.yaml
STRATEGIST_BUNDLE.md
```

```

The kernel should never execute directly from free-form Markdown.

Working Rules

Never:

- redesign existing kernel functionality without evidence
- duplicate modules
- duplicate providers
- introduce agent-to-agent runtime messaging
- bypass compliance
- bypass approval
- bypass audit logging

Always:

- inspect existing implementation
- identify owning stream
- identify reusable OSS
- identify reusable internal modules
- keep adapters thin
- keep state deterministic

Completion Criteria

A feature is complete only if it has:

- implementation
- tests
- logging
- configuration
- documentation
- error handling
- one verified end-to-end execution path

Scaffolding is not completion.

End-to-End Goal

...

Campaign Brief

↓

Nexus Sprint

↓

Approved Strategy Bundle

↓

Validation

↓

CampaignOps Import

↓

Campaign Review

↓

Lead Import

↓

Lead Scoring

↓

Compliance

↓

Approval

↓

Controlled Email Execution

↓

Reply Processing

↓

SLA

↓

Analytics

↓

Experiment Feedback

...

I also recommend creating one additional file called `ARCHITECTURE_DECISIONS.md` (ADR log). Every time you or another coding agent makes a significant architectural decision, record it there. Over time, this becomes invaluable because OpenCode (or any coding agent) can immediately understand *why* something was built a certain way instead of trying to reinterpret or refactor it.